

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (Bsc.IT Semester III)
Data Structures Practical

Practical - V

Roll No. :S040	Name:ADITYA SOPTE
Class :SY BSC IT	Batch : B2
Date of Assignment :22-07-2026	Date/Time of Submission :22-07-2026

Binary Search Tree: Write a program to:

- a. Create a binary search tree.

CODE:

```
gfdgd.py - C:/Users/IT-23/AppData/Local/Programs/Python/Python314/idle/gfdgd.py (3.14.6)
File Edit Format Run Options Window Help

class TreeNode:
    def __init__(self,data):
        self.data = data
        self.left = None
        self.right= None

root = TreeNode(0)
nodeA= TreeNode(1)
nodeB= TreeNode(2)
nodeC= TreeNode(3)
nodeD= TreeNode(4)
nodeE= TreeNode(5)
nodeF= TreeNode(6)
nodeG= TreeNode(7)

root.left = nodeA
root.right= nodeB

nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.left = nodeF

nodeA.left = nodeG
print("Node data",root.left.right.data)
```

OUTPUT:

```
= RESTART: C:/\nNode data 4
```

b. Insert nodes into a binary search tree.

CODE:

gfdfd.py - C:/Users/IT-23/AppData/Local/Programs/Python/Python314/idle/gfdfd.py (3.14.6)

```
File Edit Format Run Options Window Help\nclass TreeNode:\n    def __init__(self,data):\n        self.data = data\n        self.left = None\n        self.right= None\n\nroot = TreeNode(0)\nnodeA= TreeNode(1)\nnodeB= TreeNode(2)\nnodeC= TreeNode(3)\nnodeD= TreeNode(4)\nnodeE= TreeNode(5)\nnodeF= TreeNode(6)\nnodeG= TreeNode(7)\nnodeH= TreeNode(8)\n\nroot.left = nodeA\nroot.right= nodeB\n\nnodeA.left = nodeC\nnodeA.right = nodeD\n\nnodeB.left = nodeE\nnodeB.left = nodeF\n\nnodeA.left = nodeG\nnodeB.right =nodeH\nprint("Node data",root.right.right.data)\n|
```

OUTPUT:

```
= RESTART: C:/\nNode data 8
```

c. Search for a node in a binary search tree.

CODE:

```

*gfdfd.py - C:/Users/IT-23/AppData/Local/Programs/Python/Python314/idle/c
File Edit Format Run Options Window Help

class TreeNode:
    def __init__(self,data):
        self.data = data
        self.left = None
        self.right= None

root = TreeNode(0)
nodeA= TreeNode(1)
nodeB= TreeNode(2)
nodeC= TreeNode(3)
nodeD= TreeNode(4)
nodeE= TreeNode(5)
nodeF= TreeNode(6)
nodeG= TreeNode(7)
nodeH= TreeNode(8)

root.left = nodeA
root.right= nodeB

nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.left = nodeF

nodeA.left = nodeG
nodeB.right =nodeH
print("Node data",root.right.right.data)

def search(root, key):
    if root is None:
        return False
    print("Checking:", root.data)

    if root.data == key:
        return True

    return search(root.left, key) or search(root.right, key)

key = int(input("Enter element to search: "))

if search(root, key):
    print("Element Found")
else:
    print("Element Not Found")
    print("Aditya Sopte S040")

```

OUTPUT:

```

-----
Node data 8
Enter element to search: 7
Checking: 0
Checking: 1
Checking: 7
Element Found

```

Curiosity Questions:

1. In a BST, which side (left or right) has **smaller numbers** than the root?
2. If we insert numbers 10, 20, 5 into a BST, which number will be at the root?
3. What will the inorder traversal of a BST give — sorted numbers or random numbers?
4. Where will the **largest number** in a BST always be found?
5. If the root of a BST is 50 and we search for 70, will we go left or right? Why?
6. If a BST has only **one node**, what is its inorder traversal?
7. Can we create a BST from characters like A, B, C instead of numbers?
8. If we insert numbers 5, 10, 15, 20 in order, will the tree be balanced or like a straight line?
9. In a BST, do we check the left child first or the right child first in inorder traversal?
10. If we delete the root of a BST, will the tree disappear?